

FE Interview Flashcards — Quick Fire

Question → short answer. No explanations. For between-round review.

JavaScript Core

Hoisting? → Declarations moved to top of scope; `var` → `undefined`, `let/const` → TDZ error if accessed early.

`var` vs `let` vs `const`? → `var`=function-scoped/redeclarable; `let`=block-scoped/reassignable; `const`=block-scoped/no reassignment.

Closure? → Function that retains access to its outer scope's variables after the outer function returns.

`==` vs `===`? → `==` coerces types before comparing; `===` compares value+type strictly.

`this` in regular function? → Determined by call site: `undefined`(strict)/global, or the object before the dot.

`this` in arrow function? → Lexically inherited from enclosing scope, never its own.

`call` vs `apply` vs `bind`? → `call`=args individually+invokes now; `apply`=args as array+invokes now; `bind`=returns new bound function, invokes later.

Event loop order? → Sync code → all microtasks (Promises) → one macrotask (setTimeout) → repeat.

Microtask vs macrotask? → Microtask=Promise/queueMicrotask (higher priority); macrotask=setTimeout/setInterval/I/O.

Prototypal inheritance? → Objects link to another object via `[[Prototype]]`; property lookup walks this chain.

`null` vs `undefined`? → `undefined`=declared, not assigned; `null`=intentional "no value."

`typeof null`? → `"object"` (known JS quirk).

Pure function? → Same input → same output, no side effects.

Shallow vs deep copy? → Shallow copies top level only (nested objects shared); deep copies all levels (`structuredClone`).

Currying? → Transform `f(a,b,c)` into `f(a)(b)(c)`.

Debounce? → Delay execution until calls pause (search input).

Throttle? → Execute at most once per interval (scroll handler).

Memoization? → Cache function output by input to skip recomputation.

`NaN === NaN`? → `false`. Use `Number.isNaN()` to check.

ES6+

Destructuring default+rename? → `const { name: userName = 'x' } = obj`.

Spread vs Rest? → Spread expands `([...arr])`; Rest collects `((...args) =>)`.

Optional chaining `(?.)`? → Returns `undefined` instead of throwing on null/undefined access.

Nullish coalescing `(??)`? → Falls back only on `null`/`undefined`, not `0`/`''`/`false` (unlike `(||)`).

Generator function? → `function*` + `yield`, pausable/resumable, returns iterator.

Map vs **Object**? → `Map` = any key type, ordered, `.size`, better for frequent add/remove.

Set use case? → Store unique values, easy dedup `([...new Set(arr)])`.

ES Modules vs CommonJS? → ESM=static/tree-shakeable/async; CommonJS=`require`, sync, runtime-resolved.

Promise.all vs **allSettled**? → `all` rejects fast if any fails; `allSettled` waits for all, never rejects.

Promise.race vs **any**? → `race`=first to settle (win or fail); `any`=first to fulfill, fails only if all fail.

for...in vs **for...of**? → `for...in`=keys (avoid on arrays); `for...of`=values (use for arrays/iterables).

DSA Quick Recall

Two Sum approach? → Hashmap of `value→index`, one pass, O(n).

Valid palindrome approach? → Two pointers from both ends, skip non-alphanumeric.

Longest substring w/o repeats? → Sliding window + Map of last-seen index.

Flatten nested array? → Recursive `reduce` + `Array.isArray` check.

Group anagrams? → Hashmap keyed by sorted-string signature.

Move zeros to end? → Two-pointer in-place swap.

Balanced parentheses? → Stack; push open, pop on close, check match.

Missing number 1..n? → Sum formula $\frac{n(n+1)}{2}$ minus actual sum, or XOR trick.

Array access/search complexity? → Access $O(1)$, Search $O(n)$.

Hashmap avg complexity (all ops)? → $O(1)$ average.

React

Virtual DOM? → In-memory UI tree; diffed against previous tree to compute minimal real DOM updates.

Fiber? → React's reconciliation engine enabling pausable/prioritized incremental rendering.

Why not index as key? → Breaks item identity on reorder/insert/delete → wrong state/DOM reuse.

`useEffect` vs `useLayoutEffect`? → `useEffect`=async after paint; `useLayoutEffect`=sync before paint (for DOM measurement).

`useMemo` vs `useCallback`? → `useMemo` memoizes a value; `useCallback` memoizes a function reference.

Rules of Hooks? → Top level only (no loops/conditions); only in components/custom hooks.

Why do Rules of Hooks exist? → React tracks hooks by call order internally; conditional calls break the order.

`React.memo` failure case? → New inline object/array/function prop each render defeats shallow comparison.

Context API pitfall? → All consumers re-render on any value change, even unused parts.

Controlled vs uncontrolled input? → Controlled=React state drives value; uncontrolled=DOM manages own state via ref.

Code splitting tool? → `React.lazy()` + `Suspense` fallback.

How to optimize slow React app? → Profile → memoize components/values → virtualize lists → code-split → fix context/key issues.

Next.js

SSR? → HTML rendered per-request on server. Good for dynamic/SEO content.

SSG? → HTML built at build time, served from CDN. Good for rarely-changing pages.

ISR? → Static page regenerated in background after `revalidate` interval.

CSR? → Rendered in browser after JS loads. Good for non-SEO dashboards.

App Router default component type? → Server Component (unless `"use client"`).

Server vs Client Component? → Server=no JS shipped, direct backend access;

Client=interactive, hooks, browser APIs.

Data fetching in App Router? → `fetch()` in async Server Component; control cache via `{ cache, next: { revalidate } }`.

Next.js Middleware? → Runs at edge before request completes — auth checks, redirects, locale detection.

`next/image` benefit? → Auto responsive sizing, lazy load, modern formats, prevents layout shift.

`next/font` benefit? → Self-hosts fonts at build time, avoids render-blocking external requests.

TypeScript

`interface` vs `type`? → `interface` supports declaration merging, best for object shapes; `type` handles unions/intersections/primitives.

`Partial<T>`? → Makes all properties optional.

`Pick<T, K>`? → Selects subset of keys.

`Omit<T, K>`? → Excludes subset of keys.

`Record<K, V>`? → Object type with keys K, values V.

Type narrowing? → Refining a union type via `typeof`, `in`, or discriminated union checks.

`any` vs `unknown`? → `any` disables checks entirely; `unknown` forces narrowing before use (safer).

`never` type? → Represents a value that can never occur (always-throws function, exhaustive switch default).

Type a React component's children? → `children?: React.ReactNode`.

Generic function example use? → Reusable typed logic, e.g. `useLocalStorage<T>(key, initialValue: T)`.

Tip: skim this top to bottom once before each round. If any answer doesn't come instantly, flag it and go deep on that one topic from the main prep doc.